0 · พื้นฐาน OOP (คำนิยามที่ชอบถาม)

OOP = เขียนโปรแกรมโดยมองเป็น**วัตถุ**ที่มีข้อมูล+พฤติกรรม ต่างจาก procedural ที่มองเป็นลำดับขั้นตอน

Class = แม่แบบ/พิมพ์เขียว กำหนด attribute + method · **Object/Instance** = สิ่งที่สร้างจาก class ด้วย `new` มีค่าของตัวเอง (1 class → หลาย object)

Attribute / Field / Property = ข้อมูล (สิ่งที่มี) · **Method** = พฤติกรรม (สิ่งที่ทำได้) · **State** = ค่า attribute ณ ขณะหนึ่ง

Constructor = ชื่อเดียวกับ class, **ไม่มี return type** (แม้ void), ทำงานตอน new, ใช้กำหนดค่าเริ่มต้น, overload ได้, **ไม่ถูกสืบทอด** · ไม่เขียนเลย Java ให้ default constructor (ไม่มีพารามิเตอร์) แต่ถ้าเขียนแบบมีพารามิเตอร์แล้ว ตัวเปล่าจะ**หายไป**

this = object ปัจจุบัน (แยก field กับ parameter ชื่อซ้ำ) · `this(...)` เรียก constructor อื่นใน class เดียวกัน (บรรทัดแรก)

Reference = ตัวแปร object เก็บ**ที่อยู่** ไม่ใช่ตัว object · `a = b` → ชี้ object เดียวกัน · `null` = ไม่ชี้อะไร → เรียกเมธอด = NullPointerException · `==` เทียบ reference, `.equals()` เทียบเนื้อหา

static = เป็นของ class ใช้ร่วมกันทุก object เรียกผ่านชื่อ class (`Math.PI`) · static method ใช้ `this` ไม่ได้ · **final**: ตัวแปร=ค่าคงที่ · method=override ไม่ได้ · class=extends ไม่ได้ (เช่น String)

4 เสาหลัก OOP: **Encapsulation** ห่อหุ้ม/ซ่อนข้อมูล (private + getter/setter + validate) · **Inheritance** สืบทอด · **Polymorphism** พ้องรูป · **Abstraction** ซ่อนรายละเอียดแสดงแค่ที่จำเป็น

getter/setter: `getName()` คืนค่า · `setGpa(x)` ตั้งค่า (ใส่ if ตรวจได้) · **boolean getter** มักชื่อ `isHonor()`

บท 7 · Array & ArrayList

Array = ตัวแปรชุด เก็บข้อมูล**ชนิดเดียวกัน**หลายค่า เรียงต่อกัน อ้างด้วย **index เริ่ม 0** · **ขนาดคงที่**หลังสร้าง · array เป็น object (ตัวแปรคือ reference)

สร้าง: `int[] a = new int[5];` · `int[] a = {1,2,3};` · `new int[]{1,2,3}` · **ค่าเริ่มต้น** int 0 · double 0.0 · boolean false · char '\u0000' · String/object **null**

length = attribute (ไม่มีวงเล็บ) · index สุดท้าย = length−1 · เกินช่วง/ติดลบ → **ArrayIndexOutOfBoundsException** (runtime ไม่ใช่ compile)

for-each `for(int x : a)` = อ่านทุกตัว ไม่รู้ index แก้ค่าในarray ไม่ได้ · ต้องการ index/แก้ค่า → for ปกติ

2 มิติ = array ของ array `new int[แถว][คอลัมน์]` · `a.length`=จำนวนแถว · `a[0].length`=คอลัมน์ · `a[r][c]` · ลูปนอก=แถว ลูปใน=คอลัมน์ · **Jagged** = แต่ละแถวยาวไม่เท่ากัน → ใช้ `a[r].length`

Array of Objects = เก็บ **reference** · `new Student[3]` สร้างแค่ช่อง (null ×3) ยังไม่มี object ต้อง `new Student(..)` ทีละช่อง · ลืม → **NullPointerException**

ข้อจำกัด Array: ขนาดตายตัว · ลบ/แทรกกลางต้องเลื่อนเอง · length = ความจุ ไม่ใช่จำนวนที่ใช้จริง → จึงมี **ArrayList**

ArrayList (`java.util`) = list ขนาด**ยืดหดอัตโนมัติ** · เก็บได้แต่ **object** → primitive ต้องใช้ **Wrapper** (int→Integer, double→Double, char→Character, boolean→Boolean) · Java แปลงให้เอง = **autoboxing/unboxing** · `<Type>` = **Generics** กำหนดชนิด

เมธอด	ทำอะไร
`add(x)` / `add(i,x)`	ต่อท้าย / แทรกที่ i (ตัวหลังเลื่อนขวา)
`get(i)` · `set(i,x)`	อ่าน · แทนที่
`remove(i)` / `remove(obj)`	ลบตาม index / ตามค่า (ตัวหลังเลื่อนซ้าย)
`size()` · `isEmpty()` · `clear()`	จำนวน · ว่างไหม · ลบหมด
`contains(x)` · `indexOf(x)`	มีไหม (boolean) · ตำแหน่ง (ไม่พบ = −1)

	Array	ArrayList
ขนาด	คงที่	เปลี่ยนได้
จำนวน	`.length`	`.size()`
ชนิด	primitive/object	object (wrapper)
เข้าถึง	`a[i]`	`get(i)`
เพิ่ม/ลบ	เลื่อนเอง	add/remove

× `ArrayList<int>` · `list.length` · `get(size())` · ลบใน for-each (ConcurrentModificationException → วนถอยหลัง) · ลืม import · เทียบ String ด้วย `==` · `i<=a.length`

```
[A,B,C] add(1,"X")→[A,X,B,C] get(2)="B"
  remove(0)→[X,B,C]
Integer [10,20,30] remove(1)→[10,30] (index ไม่ใช่ค่า)
avg = (double)sum / n;   7/2=3   (double)7/2=3.5
```

บท 8 · Inheritance (การสืบทอด)

Inheritance = class ใหม่ (**subclass/child/derived**) รับ attribute+method จาก class เดิม (**superclass/parent/base**) ด้วย `extends` แล้ว**เพิ่ม/แก้**ของตัวเองได้

จุดประสงค์: แก้ปัญหาโค้ดซ้ำ (Student/Teacher มี name, age ซ้ำ → ย้ายไป Person) · **ประโยชน์**: Code reuse · Maintainability (แก้ที่แม่ที่เดียว) · Extensibility · Real-world modeling

IS-A (เป็น...ชนิดหนึ่ง) → ใช้ inheritance · **HAS-A** (มี...) → **composition** (เก็บเป็น attribute) เช่น Car has Engine, Employee has Department · ห้ามใช้ extends กับ has-a

Single inheritance: class extends ได้ **1** class (ไม่มี multiple inheritance ของ class เพื่อเลี่ยง diamond problem) · **Multi-level** ได้: Animal→Dog→Poodle (Poodle ได้ของทั้ง 2 ชั้น) · **Hierarchical**: แม่ 1 ลูกหลายตัว · ทุก class สืบจาก **Object** โดยอัตโนมัติ

สิ่งที่ไม่ถูกสืบทอด/ใช้ตรงไม่ได้: constructor · private member (มีอยู่ แต่เข้าถึงตรงไม่ได้ ต้องผ่าน getter หรือใช้ protected)

access	class	package	subclass	ทุกที่	UML
public	✓	✓	✓	✓	+
protected	✓	✓	✓	×	#
default (ไม่เขียน)	✓	✓	×*	×	~
private	✓	×	×	×	−

*subclass ต่าง package · กว้าง→แคบ: public > protected > default > private

super: `super(args)` เรียก constructor แม่ (**ต้องบรรทัดแรก**) · `super.method()` เรียกเวอร์ชันแม่ · `super.field` · ไม่เขียน → Java แทรก `super()` ให้ → ถ้าแม่ไม่มี constructor เปล่า = **compile error**

ลำดับสร้าง object: new ลูก → constructor แม่ทำงาน**ก่อน**เสมอ (ไล่จากบนสุดลงมา) → แล้วจึงของลูก

Method Overriding = ลูกเขียนเมธอดที่สืบทอดมา**ใหม่** (เปลี่ยนพฤติกรรม) · กฎ: ชื่อเดียวกัน · พารามิเตอร์เหมือนกัน (ชนิด ลำดับ จำนวน) · return type เดียวกันหรือ subtype (covariant) · access **ห้ามแคบลง** · override **ไม่ได้**: static (=method hiding), final, private, constructor

@Override = annotation ไม่บังคับ แต่ให้ compiler ตรวจ ถ้าไม่ได้ override จริง (พิมพ์ชื่อผิด/พารามิเตอร์ผิด) → compile error · ต่อยอดแม่: `super.calculatePay() + bonus` · ไม่เรียก super = แทนที่ทั้งหมด

บท 9 · Abstraction

Abstraction = แสดงเฉพาะสิ่งจำเป็น ซ่อนรายละเอียดการทำงาน (ขับรถรู้แค่พวงมาลัย/คันเร่ง ไม่ต้องรู้ ECU) · ช่วย: กำหนดพฤติกรรมร่วม · ซ่อนส่วนที่ต่าง · ลด coupling · ทำได้ 2 ทาง: **abstract class**, **interface**

Abstract class = class ที่ยังไม่สมบูรณ์ ใช้เป็นแม่ · ใส่ `abstract` · **new ไม่ได้** แต่ใช้เป็นชนิดตัวแปรได้ (`Shape s = new Circle(5)`) · มีได้ทั้ง attribute, **constructor** (ให้ลูกเรียกผ่าน super), เมธอดปกติ, abstract method, static

Abstract method = มีแต่หัว **ไม่มี body** ลงท้าย `;` บังคับลูกให้override · ห้ามเป็น private / final / static · มี abstract method ≥1 → class **ต้อง** abstract · abstract class มี abstract method **0 ตัว** ก็ได้ (เพื่อกันการ new) · ห้าม `abstract final class`

ลูกต้อง override abstract method **ครบทุกตัว** ไม่งั้นลูกต้องเป็น abstract เอง (Shape → abstract RoundShape → Circle ทำครบ)

Interface = **สัญญา (contract)** กำหนดว่าต้องทำอะไรได้ ไม่บอกวิธี · สร้างด้วย `interface` ใช้ด้วย `implements` · **new ไม่ได้** · **ไม่มี constructor** · ไม่มี instance attribute

ใน interface: method = `public abstract` อัตโนมัติ · ตัวแปร = `public static final` (ค่าคงที่ กำหนดค่าใหม่ไม่ได้) · `default` method มี body (Java 8) · `static` method เรียกผ่านชื่อinterface · `private` method (Java 9)

implements ได้**หลาย** interface (`Phone implements Camera, MusicPlayer, GPS`) · interface `extends` interface ได้หลายตัว · class extends + implements พร้อมกันได้ (**extends ก่อน**) · เมธอดที่ implement **ต้อง public** (ไม่ใส่ = weaker access error) · ต้อง implement ครบ ไม่งั้นต้องเป็น abstract

	Abstract class	Interface
keyword	abstract class / extends	interface / implements
สืบได้	1 class	หลาย interface
constructor	มี	ไม่มี
attribute	ทุกแบบ	ค่าคงที่ public static final
method	abstract + ปกติ	abstract + default/static/private
access	ใดก็ได้	abstract method = public
ความสัมพันธ์	is-a ญาติใกล้ชิด	can-do ความสามารถ (-able)
ใช้เมื่อ	แชร์ field/ctor/โค้ดพื้นฐาน	คลาสไม่เกี่ยวกันมีความสามารถร่วม / ต้องการหลายความสามารถ

Abstraction vs Encapsulation: Abstraction = **ทำอะไรได้** ซ่อนความซับซ้อน (abstract class/interface) · Encapsulation = **ปกป้องข้อมูลอย่างไร** ซ่อน/ควบคุมการเข้าถึง (private + getter/setter)

UML: «abstract» / ชื่อตัวเอียง · method `{abstract}` · «interface» · **เส้นทึบ**หัว△โปร่ง = extends (generalization) · **เส้นประ**หัว△โปร่ง = implements (realization)

บท 10 · Polymorphism (การพ้องรูป)

Polymorphism = Poly (หลาย) + Morph (รูป) · ตัวแปรชนิดเดียวอ้างถึง object ได้หลายชนิด และ**ผลลัพธ์ขึ้นกับ object จริง** · สั่งเหมือนกัน ทำต่างกัน (move: คนเดิน นกบิน ปลาว่าย)

องค์ประกอบ: Inheritance/Interface (is-a) + Overriding + **Parent reference** ชี้ object ลูก + **Dynamic binding**

	Compile-time	Runtime
ชื่อ	**Method Overloading** · static/early binding	**Method Overriding** · dynamic method dispatch · late binding
ที่ไหน	class เดียวกัน	แม่–ลูก (ต้องมี inheritance/interface)
พารามิเตอร์	**ต่าง** (จำนวน/ชนิด/ลำดับ)	**เหมือน**
return	ต่างได้ แต่**ต่างแค่ return = error**	เหมือน/covariant
ตัดสินจาก	รายการพารามิเตอร์	ชนิด object จริงตอนรัน (JVM)
จุดประสงค์	รองรับข้อมูลหลายแบบ	เปลี่ยนพฤติกรรมของลูก

กฎทอง: **ชนิดตัวแปร (reference type)** กำหนดว่า**เรียกอะไรได้** (compiler ตรวจ) · **ชนิด object จริง** กำหนดว่า**ทำงานแบบไหน** · `Animal a = new Dog(); a.guardHouse();` → compile error

Upcasting ลูก→แม่ อัตโนมัติ ปลอดภัย · **Downcasting** แม่→ลูก ต้องเขียน `(Dog) a` · ผิดชนิด → compile ผ่าน แต่ **ClassCastException** ตอนรัน · ป้องกันด้วย `instanceof` (คืน boolean, `null instanceof X` = false) · Java 16+: `if (a instanceof Dog d)`

ไม่ polymorphic: **field** (ใช้ตามชนิดตัวแปร: `Parent o=new Child(); o.name`→ของ Parent) · **static** method (hiding) · override ไม่ได้: private, final, constructor

ประโยชน์: โค้ดซ้ำน้อย · ยืดหยุ่น · ขยายง่าย (เพิ่ม Duck ไม่ต้องแก้ลูป) · ลด if-else/switch ตามชนิด · **Loose coupling** · ดูแลง่าย · **Open/Closed Principle** = เปิดให้ขยาย ปิดการแก้ไข · **Program to an interface**: รับพารามิเตอร์เป็นชนิดแม่/interface `checkout(PaymentMethod p)`

Code smell: ใช้ instanceof/if-else ตามชนิดเยอะ = ใช้ polymorphism ไม่เต็มที่ · ใช้กับ collection: `Animal[]`, `ArrayList<Shape>` วนเรียกเมธอดเดียว

วิธีไล่ผลการรัน

① new ลูก → พิมพ์ของ constructor แม่ก่อน ② เรียกเมธอด → ชนิดตัวแปรมีไหม (ไม่มี = compile error) ③ มี → ใช้เวอร์ชันของ object จริง (ถ้าลูก override) ④ เจอ super.m() → รันของแม่แล้วกลับมาต่อ ⑤ field ใช้ตามชนิดตัวแปร

`"a"+1+2`="a12" · `1+2+"a"`="3a" · `print` ไม่ขึ้นบรรทัด · `printf("%.2f%n")` ทศนิยม 2 ตำแหน่ง · `10/0` Exception แต่ `10.0/0`=Infinity · Circle r=5 → 78.54

โจทย์ถาม → ตอบ (แนวคิด/ออกแบบ)

ถ้าโจทย์ถาม...	ตอบ
คำสั่งสืบทอด class / ใช้ interface	**extends / implements**
เรียก constructor ของแม่	**super(...) บรรทัดแรก**
อ้าง field ของ object ตัวเอง	**this**
ลูกเขียนเมธอดแม่ใหม่ (พารามิเตอร์เหมือน)	**Overriding**
ชื่อเดียวกัน พารามิเตอร์ต่าง ใน class เดียว	**Overloading**
กันไม่ให้ override / ไม่ให้สืบทอด	**final method / final class**
class ที่ new ไม่ได้ ใช้เป็นแม่แบบ	**abstract class**
เมธอดไม่มี body บังคับลูกเขียน	**abstract method**
ตรวจว่า object เป็นชนิดใด	**instanceof**
ซ่อนข้อมูล ให้เข้าถึงผ่านเมธอด	**Encapsulation (private)**
ให้ลูกเข้าถึงได้แต่คนนอกไม่ได้	**protected (#)**
ค่าคงที่ใช้ร่วมกันทุก object	**static final**
แม่ชี้ลูก JVM เลือกเมธอดตอนรัน	**Dynamic binding**
เพิ่มคลาสใหม่ได้โดยไม่แก้โค้ดเดิม	**Open/Closed Principle**
เก็บข้อมูลชุดที่จำนวนเปลี่ยนได้	**ArrayList**
wrapper ของ int / char	**Integer / Character**
จำนวนแถว / คอลัมน์ array 2 มิติ	**a.length / a[0].length**
Car กับ Engine	**HAS-A (composition)**
Circle, Rectangle มี color ร่วม แต่สูตรพื้นที่ต่างกัน	**abstract class Shape**
Bird กับ Airplane ไม่เกี่ยวกันแต่บินได้ทั้งคู่	**interface Flyable**
จ่ายเงินหลายช่องทาง เพิ่มช่องทางใหม่ง่าย	**interface PaymentMethod + polymorphism**

ทายผลเร็ว (รันจริงแล้ว)

โค้ด	ผล
`new int[3]`→a[1] · `new String[2]`→s[0]	0 · null
[A,B,C] add(1,"X") → get(2), size()	B, 4
{{75,80,70},{88,70,95},...} s[1][2]	95
A obj=new B(); obj.hi() (ctor พิมพ์ A/B, hi ของ B เรียก super.hi())	A B hiA hiB
Parent o=new Child(); o.name / o.showName()	Parent / Child method
Animal a=new Cat(); Dog d=(Dog)a;	ClassCastException
Animal a=new Dog(); a.guardHouse();	compile error
Shape[] {Circle(5), Rect(4,6), Tri(8,3)} รวมพื้นที่	114.54

บท 11 · Exception Handling

Exception = เหตุการณ์ผิดปกติ**ขณะรัน** ที่ขัดจังหวะการทำงานปกติ · ไม่จัดการ → โปรแกรมหยุด บรรทัดหลังไม่ทำ แสดง `Exception in thread "main" java.lang.ArithmeticException: / by zero`

ข้อผิดพลาด 3 แบบ: **Syntax/Compile error** ผิดไวยากรณ์ (ลืม ; ชื่อผิด) compile ไม่ผ่าน · **Runtime error** รันแล้วพัง (หาร 0, index เกิน) ← Exception · **Logical error** รันได้ผลผิด (สูตรผิด) ไม่มี error แจ้ง

Exception handling = ไม่ได้ทำให้ error หายไป แต่ทำให้โปรแกรม**รู้วิธีรับมือ** และทำงานต่อได้

Exception	สาเหตุ
ArithmeticException	หาร **int** ด้วย 0 ("/ by zero")
ArrayIndexOutOfBounds Exception	index <0 หรือ ≥ length
NumberFormatException	แปลง String ที่ไม่ใช่ตัวเลข parseInt("ABC"), parseDouble("")
NullPointerException	ใช้ reference ที่เป็น null
InputMismatchException	Scanner nextInt() แต่ผู้ใช้พิมพ์อักษร (java.util)
ClassCastException	downcast ผิดชนิด
IllegalArgumentExceptio n	อาร์กิวเมนต์ไม่ถูกต้อง (มัก throw เอง)
FileNotFound / IOException	เปิด/อ่านไฟล์ไม่ได้ (**checked**)

ลำดับชั้น: **Throwable** → **Error** (ร้ายแรงระดับระบบ เช่น OutOfMemoryError, StackOverflowError ไม่ควรจับ) | **Exception** → **RuntimeException** (unchecked) + อื่น ๆ (checked) · `catch(Exception e)` จับลูกทุกตัวได้เพราะ polymorphism

Unchecked = RuntimeException และลูก · compiler **ไม่บังคับ** จัดการ · มักเกิดจากบักโปรแกรมเมอร์ · **Checked** = Exception ที่ไม่ใช่ Runtime · compiler **บังคับ** try-catch หรือ throws ไม่งั้น compile error

try = โค้ดที่อาจเกิด exception · **catch** = จัดการเมื่อเกิด (เลือกตัว**แรก**ที่ชนิดตรง ทำตัวเดียว) · **finally** = **ทำเสมอ** ไม่ว่าเกิดหรือไม่ (ปิดไฟล์/คืนทรัพยากร) · try ต้องตามด้วย catch หรือ finally อย่างน้อย 1

Flow: try ปกติ → ข้าม catch → finally → ต่อ · เกิด exception → **ข้ามบรรทัดที่เหลือใน try** → catch ที่ตรง → finally → ต่อ · ไม่มี catch ตรง → finally → โปรแกรมพัง

Multiple catch: เรียง**เจาะจง→กว้าง** (ลูกก่อนแม่) · Exception ไว้ท้ายสุด · สลับ = compile error (unreachable) · `catch(A | B e)` จับหลายชนิดในบล็อกเดียว

ตัวแปร e: `e.getMessage()` → "/ by zero" · `System.out.println(e)` → "java.lang.ArithmeticException: / by zero" · `e.printStackTrace()` → debug

throw = **โยน** exception ทันที ในเนื้อเมธอด `throw new IllegalArgumentException("Age cannot be negative");` (ใช้ป้องกันข้อมูลผิด = encapsulation) · **throws** = **ประกาศ**ที่หัวเมธอดว่าอาจโยน ให้ผู้เรียกจัดการ `void readFile() throws IOException`

Custom exception: `class InsufficientBalanceException extends Exception` (checked) หรือ extends RuntimeException (unchecked) · constructor `super(message)` · ใช้: `if(amount>balance) throw new …("Insufficient balance");` + ประกาศ `throws`

```
try { int[] x={10,20,30}; println(x[5]); int
   r=10/0; }
catch(ArithmeticException e){println("A");}
catch(ArrayIndexOutOfBoundsException e)
   {println("I");}
finally{println("F");}  println("End");    // I F
   End
Scanner: 10,2→Result = 5 · 10,0→Cannot divide by
   zero
         ABC→Please enter a number (ไม่ถามเลขที่ 2)
```

บท 12 · Java GUI (Swing)

GUI = ติดต่อผู้ใช้ผ่านหน้าต่าง ปุ่ม ช่องกรอก (ต่างจาก **CLI**/console ที่ใช้ Scanner) · **AWT** = ชุดแรก (java.awt, heavyweight ใช้ของ OS) · **Swing** = พัฒนาจาก AWT (javax.swing, ขึ้นต้น J, component มากกว่า, มีใน JDK, OOP + event-driven) · **JavaFX** = ใหม่ รองรับ CSS/animation

Container = ใส่ component อื่นได้: **JFrame** (หน้าต่างหลัก, top-level) · **JPanel** (จัดกลุ่ม/ซ้อน layout) · **Component**: JLabel แสดงข้อความ/รูป · JTextField 1 บรรทัด (`JTextField(20)` 20=ความกว้างคอลัมน์ ไม่ใช่อักษรสูงสุด) · JPasswordField · JTextArea หลายบรรทัด (ต้องใส่ JScrollPane ถึงเลื่อนได้) · JButton · **JCheckBox เลือกได้หลายข้อ** · **JRadioButton เลือกข้อเดียว ต้องใส่ ButtonGroup** · JComboBox dropdown · JList · JTable ตาราง (ใส่ JScrollPane จึงเห็นหัวตาราง) · JOptionPane popup · JMenuBar→JMenu→JMenuItem · ImageIcon

JFrame: `setTitle` · `setSize(กว้าง,สูง)` · `setDefaultCloseOperation(EXIT_ON_CLOSE)` ปิดแล้วจบโปรแกรม (ค่าเริ่มต้นแค่ซ่อน HIDE_ON_CLOSE) · `setLocationRelativeTo(null)` กลางจอ · `setVisible(true)` **ท้ายสุด** (ค่าเริ่มต้นมองไม่เห็น) · `pack()` ปรับขนาดตาม component · `dispose()` ทำลาย/คืนหน่วยความจำ vs `setVisible(false)` แค่ซ่อน

EDT (Event Dispatch Thread) = thread ที่สร้าง/แก้ GUI → `SwingUtilities.invokeLater(() -> …)` · งานหนักบน EDT = หน้าจอค้าง → ใช้ SwingWorker

Layout Manager = กฎจัดตำแหน่ง/ขนาด component · **FlowLayout** เรียงซ้าย→ขวา เต็มขึ้นบรรทัดใหม่ (ค่าเริ่มต้นของ **JPanel**, ฟอร์มเล็ก) · **BorderLayout** 5 ส่วน NORTH/SOUTH/EAST/WEST/CENTER (ค่าเริ่มต้นของ **JFrame**, ไม่ระบุ = CENTER, ใส่ซ้ำช่องเดิม = ทับ) · **GridLayout(r,c,hgap,vgap)** ตารางช่องเท่ากัน เติมทีละแถว · **BoxLayout** แนวเดียว · **GridBagLayout** ยืดหยุ่นสุด · ใช้ JPanel ผสมหลาย layout · **ไม่แนะนำ null layout + setBounds** (ไม่ยืดหยุ่น เปลี่ยนเครื่อง/ฟอนต์แล้วเพี้ยน)

Event-Driven Programming = โปรแกรม**รอ**ผู้ใช้กระทำ (Event) แล้วตอบสนอง ไม่ได้รันบนลงล่างจบ · Action → Listen → Respond · **Event Source** (JButton) → **Event Object** (ActionEvent) → **Event Listener** (ActionListener)

Listener	ใช้กับ / เมธอด
ActionListener	คลิกปุ่ม/Enter/เมนู · `actionPerformed(ActionEvent e)` · มีเมธอดเดียว → ใช้ **lambda** `e -> …`
ItemListener	เปลี่ยนตัวเลือก ComboBox/CheckBox · `ItemEvent.SELECTED`
MouseListener / MouseAdapter	คลิก/เมาส์ `mouseClicked`
KeyListener / KeyAdapter	คีย์บอร์ด `keyReleased` (DocumentListener ดีกว่า จับการวางด้วยเมาส์)

Adapter class = implement listener ไว้แล้ว (เมธอดว่าง) → override เฉพาะที่ใช้ ประหยัดโค้ด · ไม่ `addActionListener` = กดแล้วไม่เกิดอะไร · import `java.awt.event.*`

JOptionPane: `showMessageDialog` แจ้งเตือน (void) · `showInputDialog` รับข้อความ คืน **String / null ถ้ากด Cancel** · `showConfirmDialog(…, YES_NO_OPTION)` คืน **int** เทียบ `YES_OPTION` · ชนิดไอคอน: INFORMATION · WARNING · ERROR · QUESTION · PLAIN _MESSAGE

ค่าที่ได้: `getText()` คืน **String** เสมอ → คำนวณต้อง `Integer.parseInt` / `Double.parseDouble` + `.trim()` + catch NumberFormatException · `isSelected()` (check/radio) · `(String) getSelectedItem()` / `getSelectedIndex()` (combo) · `setEditable(false)` แก้ไม่ได้ · `requestFocus()` · **DefaultTableModel** addRow/removeRow, `getSelectedRow()` = −1 ถ้าไม่เลือก

MVC / แยก Model ออกจาก GUI: Model = คลาสเก็บข้อมูล+logic ไม่รู้จัก UI · GUI = รับข้อมูล → validate → สร้าง object Model → แสดงผล · ข้อดี: อ่านง่าย ทดสอบง่าย นำ Model ไปใช้กับ console/web ได้ · Polymorphism ใน GUI: `Shape s = combo=="Circle" ? new Circle(r) : new Rectangle(w,h); s.calculateArea();`

หลัก UX: จัดกลุ่มด้วย JPanel+Border · label สื่อความหมาย · error ชัดเจน · confirm ก่อนลบ/ปิด · validate ข้อมูล · **แยก GUI จาก logic** · validation: ไม่ครบ → showMessageDialog(WARNING) แล้ว `return;`

ข้อความถูก/ผิดที่ชอบออก

- ✓ abstract class มี constructor ได้ · ✗ interface มี constructor
- ✓ abstract class ไม่มี abstract method เลยก็ได้ · ✗ new abstract class/interface ได้
- ✓ ตัวแปรชนิด interface/abstract ชี้ object ลูกได้ · ✓ implements ได้หลาย interface
- ✗ extends ได้หลาย class · ✓ interface extends หลาย interface ได้
- ✗ constructor ถูกสืบทอด/override ได้ · ✓ constructor overload ได้
- ✗ overloading ต่างแค่ return type ได้ · ✓ overloading = compile-time, overriding = runtime
- ✗ override แล้วลด access ได้ · ✗ override static/final/private ได้
- ✓ super() ต้องบรรทัดแรก · ✓ constructor แม่ทำก่อนลูก
- ✓ downcast ผิดชนิด compile ผ่านแต่ ClassCastException · ✗ field เป็น polymorphic
- ✓ finally ทำงานเสมอ · ✗ หลังเกิด exception บรรทัดที่เหลือใน try ยังทำ
- ✓ checked ต้อง try-catch/throws · ✗ ArithmeticException เป็น checked
- ✗ array เปลี่ยนขนาดได้ · ✗ ArrayList เก็บ int โดยตรง · ✓ new Student[3] ได้ null 3 ช่อง
- ✓ JRadioButton ต้อง ButtonGroup · ✓ getText() คืน String · ✓ showInputDialog กด Cancel = null
- ✓ JPanel ค่าเริ่มต้น FlowLayout, JFrame = BorderLayout · ✗ JTextField(20) = รับได้ 20 ตัวอักษร

โจทย์ถาม → ตอบ (บท 11–12)

ถ้าโจทย์ถาม...	ตอบ
บล็อกที่ทำงานเสมอ	**finally**
catch ที่จับได้ทุก exception	**catch (Exception e)**
คลาสบนสุดของ error ทั้งหมด	**Throwable**
โยน exception เอง / ประกาศที่หัวเมธอด	**throw / throws**
สร้าง exception ของตัวเอง	**extends Exception**
compiler บังคับให้จัดการ	**Checked (IOException)**
มี return ใน try แล้ว finally?	**finally ยังทำก่อน return**
ไม่เกิด exception: catch ทำไหม	**ข้าม catch, ทำ finally**
หน้าต่างหลัก / จัดกลุ่ม component	**JFrame / JPanel**
เลือกได้ข้อเดียว / หลายข้อ	**JRadioButton+Button Group / JCheckBox**
dropdown / หลายบรรทัด / รหัสผ่าน	**JComboBox / JTextArea / JPasswordField**
แบ่ง 5 ทิศ / ตารางเท่ากัน / ซ้าย→ขวา	**Border / Grid / FlowLayout**
เมธอดที่ทำงานเมื่อคลิกปุ่ม	**actionPerformed(ActionEvent e)**
ผูกปุ่มกับงาน	**addActionListener**
package ของ ActionListener / Swing	**java.awt.event / javax.swing**
อ่าน / เขียนข้อความใน field	**getText() / setText()**
popup แจ้งเตือน / ถาม Yes-No / รับค่า	**showMessage / showConfirm / showInputDialog**
ปิดหน้าต่างแล้วจบโปรแกรม	**EXIT_ON_CLOSE**
หน้าต่างอยู่กลางจอ	**setLocationRelativeTo (null)**
ตารางเพิ่ม/ลบแถวได้	**DefaultTableModel**
thread ที่จัดการ GUI	**EDT (invokeLater)**
แยกข้อมูลออกจากหน้าจอ	**MVC (Model กับ View)**

แม่แบบตอน 2 (ย่อ)

```
public abstract class Pet {                    //
   UML: # protected
  protected String name;
  public Pet(String name){ this.name = name; }
  public abstract void makeSound();
  public void showPetInfo(){
     System.out.print(name+" Making Sound : ");
     makeSound(); }
}
public class Dog extends Pet {
  public Dog(String name){ super(name); }
  @Override public void makeSound(){
     System.out.println("Bok Bok..."); }
}
ArrayList<Pet> pets = new ArrayList<>();
   pets.add(new Dog("Buddy"));
for (Pet p : pets) p.showPetInfo();
// Employee: abstract double calculateSalary();
   FullTime=base+bonus, PartTime=hours*rate
calcBtn.addActionListener(e -> calculate());
private void calculate(){
  try {
    double a =
       Double.parseDouble(firstField.getText().trim
       ());
    double b =
       Double.parseDouble(secondField.getText().tri
       m());
    resultLabel.setText("ผลลัพธ์ = " + (a + b));
  } catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "กรุณากรอก
       ข้อมูลเป็นตัวเลข",
         "ข้อมูลไม่ถูกต้อง", JOptionPane.ERROR_MESSAGE);
  }
}
setLayout(new BorderLayout(10,10)); add(formPanel,
   BorderLayout.NORTH);
JPanel formPanel = new JPanel(new
   GridLayout(2,2,5,5));
SwingUtilities.invokeLater(() -> new
   AdditionGUI().setVisible(true));
```